Lab No. 03
Managing Processes, and Writing, Compiling & Executing C on Linux and GDB

Objective

To provide practical experience in basic Linux process management and C programming, focusing on command-line compilation, execution, and debugging with GDB.

Outcomes

Upon completion, students will be able to:

1. Manage Linux processes using commands like ps, top, kill, nice, and renice.
2. Write, compile, and execute simple C programs from the command line using GCC.
3. Debug C programs effectively with GDB, including setting breakpoints and inspecting variables.

Viewing Running Processes: The Tools You Need

Knowing how to view and interpret running processes is crucial. Several tools in Linux allow you to monitor active processes in real-time. Here's a breakdown of the most effective ones.

1. ps — Snapshot of Running Processes

The `ps` command is a simple yet powerful tool for capturing a snapshot of the current processes. It provides key information like the Process ID (PID), CPU usage, memory consumption, and more.

- Common Commands:
- `ps aux`: Lists all processes running on the system.
- `ps -ef`: Shows a full-format listing with more detailed info.

2. top — Real-Time System Monitoring

The top command provides a dynamic, real-time view of system processes, including CPU and memory usage. It's a go-to tool for administrators to monitor system health.

- Usage: Just type top and watch as processes update every few seconds.

- Interactive Commands:
- P: Sort by CPU usage.
- M: Sort by memory usage.
- k: Kill a process by PID.
- q: Exit top.

3. htop — Interactive Process Viewer

`htop` is a more advanced version of `top`, with a user-friendly interface. It allows you to interact with processes in a more visual manner, making it easier to navigate through the system.

Installation: `sudo apt install htop`

- Run: `htop`

Features: Colored output, interactive process management, and a tree-like process structure.

4. pgrep — Search for Processes by Name

If you're looking to find a specific process without knowing its PID, `pgrep` allows you to search for processes by name or other criteria.

- Example: `pgrep nginx` – Find the PID of an nginx process.

Managing Processes: Taking Control of Your System

Once you know how to identify processes, the next step is learning how to manage them. This includes terminating, pausing, or adjusting the priority of processes to ensure smooth operation.

How to Kill a Process in Linux

There are several ways to kill a process in Linux, but the first step is always to identify the process you want to kill. Once you identify the process, you can choose the following method based on your needs:

1. Kill Command With a PID

Finding the Process ID (PID): To kill a process, you must first know its Process ID (PID). The ps command, combined with grep, helps you find this. Open your terminal and type:

- `ps aux | grep process_name`

Replace “process_name” with the program name you want to terminate. For example, if you’re looking for a Firefox process, you might type:

- Run: `ps aux |grep firefox`

This command will display a list of processes matching the name. The second column in the output shows the PID.

- Run: `ps aux |grep nano`

Using the kill command: Once you have the PID, you can use the kill command to terminate the process:

- `ps kill PID`

Replace “PID” with the actual number you found. For instance:

- Run: `ps kill 1234`

Using kill with multiple PIDs: If you know the PIDs of all processes you want to terminate, you can list them after the kill command:

- `kill PID1 PID2 PID3`

For example:

- Run: `kill 1234 5678 9101`

2. **Kill by Name (pkill):**

The simplest method is by running the following command:

- Run:`pkill process_name`

Don’t forget to replace process_name with the name of the process you are trying to kill.Use the -f option to match a substring in the process name for more flexible matching.

- `pkill -f "partial_process_name"`

For example,`pkill -f "firefox"` would match any process with “firefox” anywhere in its command line.

3. **Killall Command**

The killall command is similar to `pkill` but requires an exact match of the process name:

Basic usage: To kill all processes with an exact name match:

- `killall process_name`

Forceful termination: For stubborn processes use the -9 option (equivalent to SIGKILL):

- `killall -9 process_name`

4. renice — Adjust Process Priority

To prevent system overloads, you can adjust the priority of processes using the `nice` and `renice` commands.

- **nice:** Start a process with a specified priority.
- Example: `nice -n -10 command_name` – Start a process with higher priority.
- **renice:** Change the priority of an existing process.
- Example: `renice -n 10 -p 1234` — Change the priority of a running process.

Background and Foreground Processes: Managing Process Execution

In Linux, you can run processes in the background or foreground, depending on how you want to interact with the system.

1. Running a Process in the Background

To free up your terminal, you can run commands in the background by appending an ampersand (&) to the command.

- Example: `command_name &`

2. **Bringing a Background Process to the Foreground**

Use the fg command to bring a background process back to the foreground.

- Example: `fg %1` – Bring the first background job to the foreground.

3. **Suspending and Resuming a Process**

You can suspend a running process and send it to the background using the `Ctrl+Z` shortcut. To resume it, use the bg command.

- Example: `bg %1` – Resume the suspended process in the background.

Analyzing Process Resources: Monitoring System Health

To keep your system running efficiently, it's essential to analyze how much system resources your processes are using.

1. iotop — Monitor Disk I/O

If you're dealing with disk-intensive processes, `iotop` allows you to monitor disk I/O activity.

- Installation: `sudo apt install iotop`

Run: `sudo iotop` – View real-time disk I/O by processes.

Simulating a High CPU Resource Process

create a process that consumes significant CPU so we can observe its behavior and practice monitoring, priority adjustment, and termination.

- Run: `yes > /dev/null &`

Explanation:
`yes` is a Linux command that continuously outputs `y` to the screen.`> /dev/null` redirects this output to a special file that discards it, preventing terminal spam.
`&` sends the command to the background, allowing you to keep using the terminal.
The result: a process that runs an infinite loop, using almost 100% CPU on one core.

Graded Task :

Scenario 1:
A background process has gone rogue and is hogging CPU resources. Your task is to find it and safely terminate it without affecting essential system processes.

What to do:

1. Use `top` or `ps` to identify the process using the most CPU.
2. Confirm that it's safe to kill (not a system-critical process).
3. Use `kill` or `kill -9` to terminate it.
4. Capture screenshots or terminal output before and after termination.

Scenario 2:
Your system is running multiple data-heavy tasks, and some are slowing everything down. You need to adjust process priorities and monitor disk I/O to ensure smoother system performance.

Bonus Task:

Simulate Multiple Background Jobs & Manage Them (**Hint:**Run several background processes, then suspend, resume, and terminate selectively.)

Section 2: Writing, compiling, and executing C programs

Objective

- To understand the basic workflow of writing, compiling, and executing C programs on Linux.
- To use GCC for compilation and GDB for debugging.
- To learn how to troubleshoot and debug a simple C program using GDB.

Why Use GDB?

GDB provides several advantages for debugging programs:

- Breakpoints and Watchpoints: Stop execution at specific lines or when a variable changes.
- Step Through Code: Execute code line-by-line to examine flow and logic.
- Backtrace and Stack Inspection: Analyze the state of function calls leading to a crash.
- Modify Variables at Runtime: Change variable values without restarting the program.

- Disassemble Code: View assembly instructions for low-level debugging.

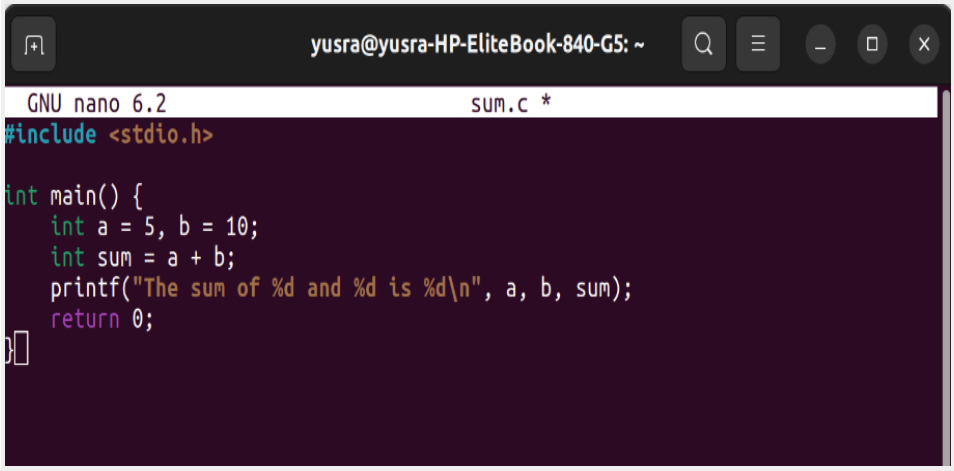
Part 1: Environment setup

Before starting, you need to ensure the GNU Compiler Collection (GCC) and the GNU Debugger (GDB) are installed on your Linux system. For Debian-based systems like Ubuntu, follow these steps:

1. Open a terminal. You can typically do this by pressing `Ctrl + Alt + T` or by searching for "Terminal" in your applications menu.
2. Update your package lists: It's good practice to update your system's package information before installing new software.
    - `sudo apt update`
3. Install the build-essential package: This package includes the GCC compiler, GDB, and other tools necessary for building software. It is the recommended way to get started quickly.
    - `sudo apt install build-essential gdb`
4. Verify the installation: You can check that both GCC and GDB are correctly installed by running a version check.
    - For GCC: `gcc --version`
    - For GDB: `gdb –version`

Part 2: Writing, compiling, and executing C code

1. **Write the C program:**
   Open a terminal and create a new C file: `nano sum.c`



Save and exit the text editor:In Nano, press Ctrl + X, then press Y to save and Enter to confirm.

2. **Compile with GCC:**

The gcc command compiles your source code. The -o flag specifies the name of the output executable file.

Run the following command in your terminal:

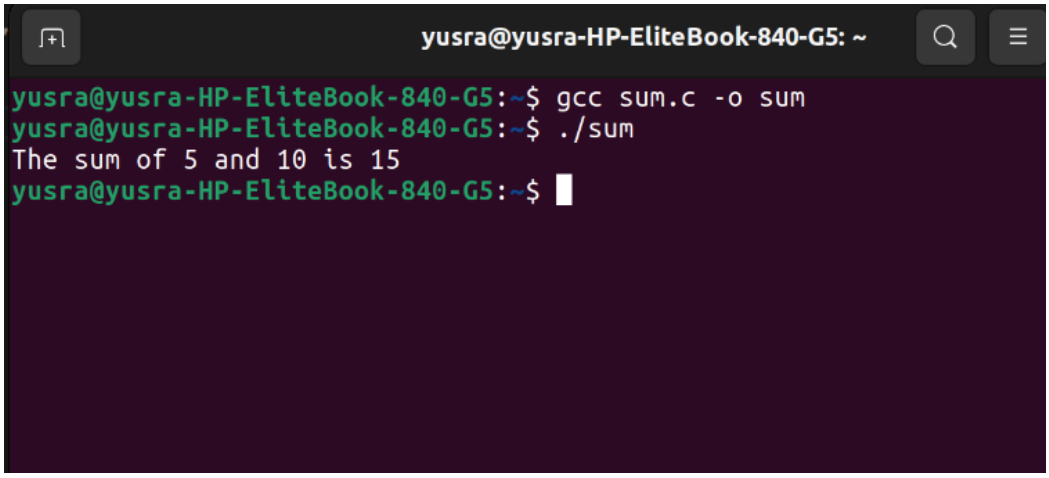
- gcc hello.c -o hello

hello.c: The source code file.
-o hello: Specifies the output file name (hello), which is the executable.

Check if the compilation was successful:

If there are no errors, the terminal prompt will return without any messages.

3. **Execute the program:**
    - ./hello



Part 3: Debugging with GDB

1. **Recompile for debugging:**

   GDB requires special debugging symbols to be included in the executable. You must recompile your program using the `-g` flag.
    - gcc -g -o example example.c
2. **Start the debugger:**

   Launch GDB with your executable file as an argument.

   The prompt will change to `(gdb)` once the debugger is ready.
    - gdb ./example

3. **Basic GDB commands:**

Command	Explanation	Example
`break <line_number>` (`b`)	Set a breakpoint. The program will pause execution at the specified line.	`(gdb) break 6`
`run` (`r`)	Starts the program execution within the debugger.	`(gdb) run`
`print <variable_name>` (`p`)	Displays the value of a variable at the current breakpoint.	`(gdb) print a`
`next` (`n`)	Executes the next line of code, stepping over any function calls.	`(gdb) next`
`step` (`s`)	Executes the next line of code, stepping into any function calls to debug them line by line.	`(gdb) step`
`continue` (`c`)	Resumes normal program execution until the next breakpoint is hit or the program exits.	`(gdb) continue`
`list` (`l`)	Shows the source code around the current line of execution.	`(gdb) list`
`info breakpoints` (`info b`)	Lists all currently set breakpoints, showing their numbers, status (enabled/disabled), and locations.	`(gdb) info breakpoints`

disable <bp_number> (dis)	Disables a specific breakpoint by its number. The breakpoint remains set but is inactive.	(gdb) disable 1
enable <bp_number> (en)	Re-enables a previously disabled breakpoint by its number.	(gdb) enable 1
delete <bp_number> (d)	Permanently deletes a breakpoint by its number. If no number is specified, it will prompt to delete all.	(gdb) delete 2
finish	Executes the rest of the current function and stops when it returns to the calling function.	(gdb) finish
watch <expression>	Sets a watchpoint that stops the program whenever the value of a variable or expression changes.	(gdb) watch my_variable
set variable <var>=<value>	Allows you to change the value of a program variable during a debug session to test different scenarios.	(gdb) set variable a = 20
backtrace (bt)	Displays a stack trace of all active function calls, showing the path taken to reach the current execution point.	(gdb) backtrace
quit (q)	Exits the GDB debugger.	(gdb) quit

4. **Debugging exercise:**

   Set a breakpoint at line 6, then run the program:

   - (gdb) break 6

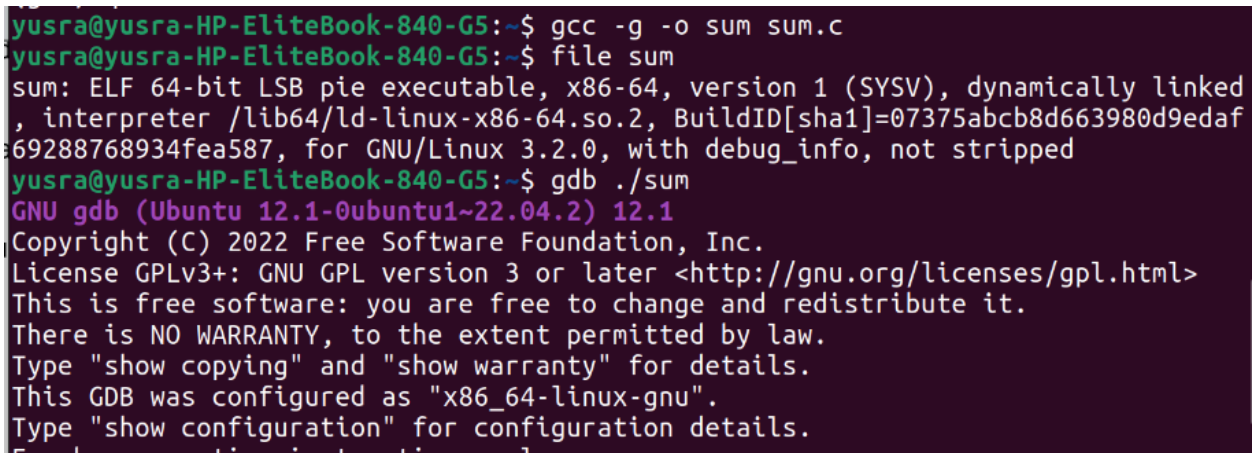
- (gdb) run

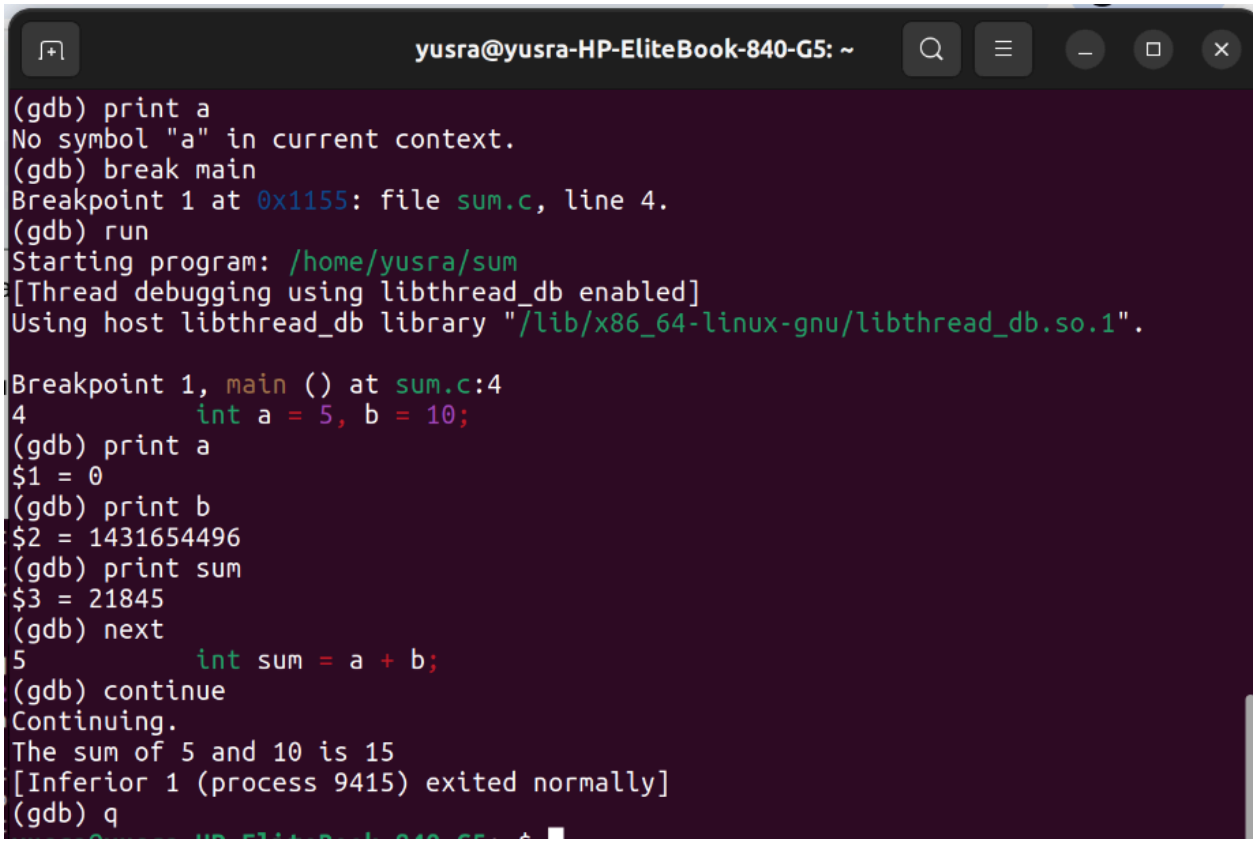
When the program pauses, inspect the values of `a` and `b` to confirm they are correct:

- (gdb) print a
- (gdb) print b

Use the `next` command to step over the `sum` calculation, and then print `sum` to verify the result before continuing execution:

- (gdb) next
- (gdb) print sum
- (gdb) continue





Explanation of the Output:

1. Breakpoint at Line 4:
   The program pauses at line 4, where variables a and b are declared, but not yet initialized. At this point, GDB prints:
   - a = 0
   - b = 1431654496
   - sum = 21845

2. These values are not what you expected because they are values in memory before the variables are actually assigned.

3. After Executing the Line `int sum = a + b;`:
   Once you execute the `next` command, the variables are initialized properly, and the program executes line 5:

   - `sum = a + b` should evaluate `to` `sum = 5 + 10 = 15.`

4. Continue Execution:
   The program finishes, prints `The sum of 5 and 10 is 15`, and exits.

Activities:

1. Modify the `practice` program to accept two integer inputs from the user instead of using hard-coded values.
2. The program should then print the product of the two numbers.
3. Use GDB to debug your new program. Set a breakpoint after the user input is read and verify that the values stored in your variables are correct.

Reference Learning:

- https://medium.com/havingfun/debugging-c-code-with-gdb-90adb2f3da96
- https://medium.com/@saransivakumar89/a-beginners-guide-to-gdb-the-gnu-debugger-57bd6e01e9e4
- https://medium.com/@seantywork/get-owned-by-multithreaded-c-debugging-or-own-it-with-gdb-tips-for-go-included-2ab008484720

Graded Task:

Task1:
You are provided with a C program that calculates the factorial of a number provided as a command-line argument. However, the program contains logical bugs and potential overflow issues.
Your job is to:

1. Compile and run the program.
2. Use GDB to step through the code and observe where things go wrong.
3. Fix the issues and provide the corrected version of the program.

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int factorial = 1;
    int number;

    if (argc != 2) {
        printf("Usage: %s <number>\n", argv[0]);
        return 1;
    }

    number = atoi(argv[1]);

    for (int i = 1; i < number; i++) {
        factorial *= i;
    }

    factorial *= number;

    printf("The factorial of %d is: %d\n", number, factorial);

    return 0;
}
```

Checkpoint Questions

1. What happens if you omit **-g** during compilation?
2. Why does the original code produce **wrong results for large numbers**?
3. How did you confirm the **off-by-one error** using GDB?

Bonus Challenge

Modify the program so that:

- It handles factorials up to 20 correctly using unsigned long long.
- It prints an error message if the input is negative or too large (e.g., > 20).

Task 2:

Instructions

1. Compile the program and observe the errors or warnings generated by the compiler.
2. Run the program and note any abnormal behavior (e.g., crashes, incorrect outputs).
3. Use GDB (GNU Debugger) to:
    - Set breakpoints at important points (loops, sum calculation).
    - Inspect variable values during execution.
    - Identify segmentation faults or unexpected values.
4. Fix the bugs one by one, ensuring that:

- The program no longer crashes.
- The output is correct for various input values (e.g., n = 3 with values 1, 2, 3 should output 6).

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, i;
    int *arr;
    int sum = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    arr = malloc(n * sizeof(int));

    printf("Enter %d integers:\n", n);
    for (i = 0; i <= n; i++) {
        scanf("%d", arr + i);
    }

    for (i = 0; i < n; i++); {
        sum += arr[i];
    }

    printf("Sum = %d\n", sum);

    free(arr);
    return 0;
}
```

Submission Instructions:

Submit a report that includes:

1. **Terminal Screenshots**:
    - Show all major steps of process management tasks (e.g., finding CPU hogging process, adjusting priorities, terminating processes).
    - Include **before and after states** for each action.
2. **Explanations for Each Screenshot**:
    - Describe what command you ran and why.
    - Mention the expected outcome and what you observed.
3. **Code & Debugging Evidence** (for C debugging tasks):
    - Include the **original buggy code** and your **corrected version**.
    - Add a brief note on what the bug was and how you fixed it.
    - If you used GDB, include the commands you used (breakpoints, print statements, etc.) and the observed output.
4. **Final Output**:
    - Show the correct result after fixing the issue or completing the task.